Personal Weather Intelligence & Forecasting System

Technical Product Specification

Build a self-hosted, locally running weather intelligence and forecasting platform designed around a single primary objective:

> Produce the best possible short- and medium-range weather forecast for the Dallas–Fort Worth area while maintaining a continuously updated, highly detailed view of U.S. and Atlantic tropical weather.

This should not merely be a weather dashboard. It should be designed as a personal weather research/forecasting laboratory capable of ingesting real-time atmospheric observations and operational model output, running locally available numerical/AI forecasting models on NVIDIA hardware, performing ensemble/model comparison, and eventually learning where existing models systematically fail in the DFW region.

The system should be designed so that the forecasting engine is independent of the UI and can operate autonomously in the background.

1. Hardware Target

Primary machine:

- CPU: AMD Ryzen 7 7700X
- GPU: NVIDIA RTX 5070 Ti
- GPU VRAM: 16 GB
- System RAM: 64 GB DDR5
- Local SSD storage
- NVIDIA CUDA environment

The system MUST prioritize local GPU computation wherever practical.

The architecture should support:

- PyTorch
- JAX
- CUDA
- GPU-accelerated numerical computation

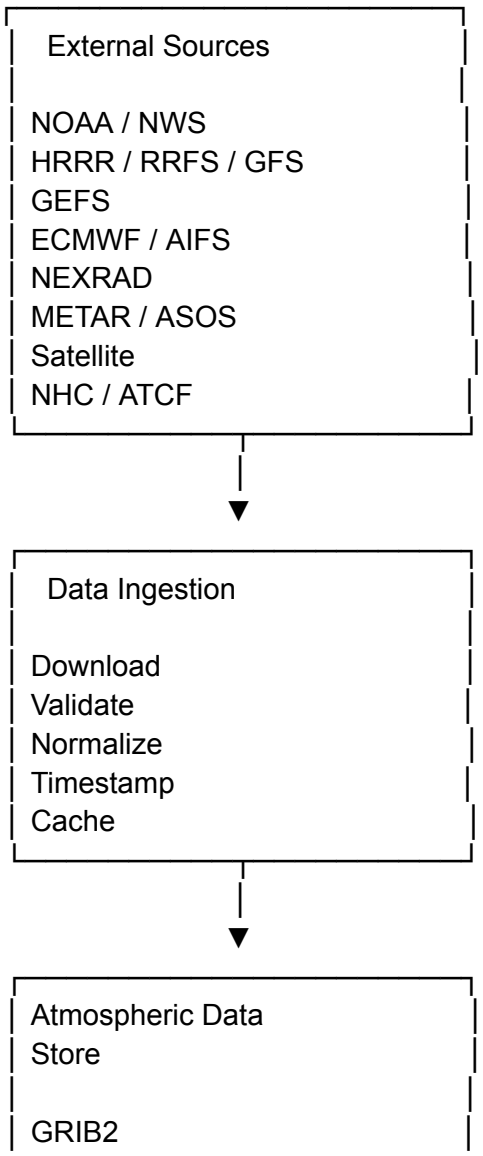
- GPU inference for pretrained weather/AI models
- CPU parallelism for data processing
- asynchronous data ingestion
- local caching of large datasets

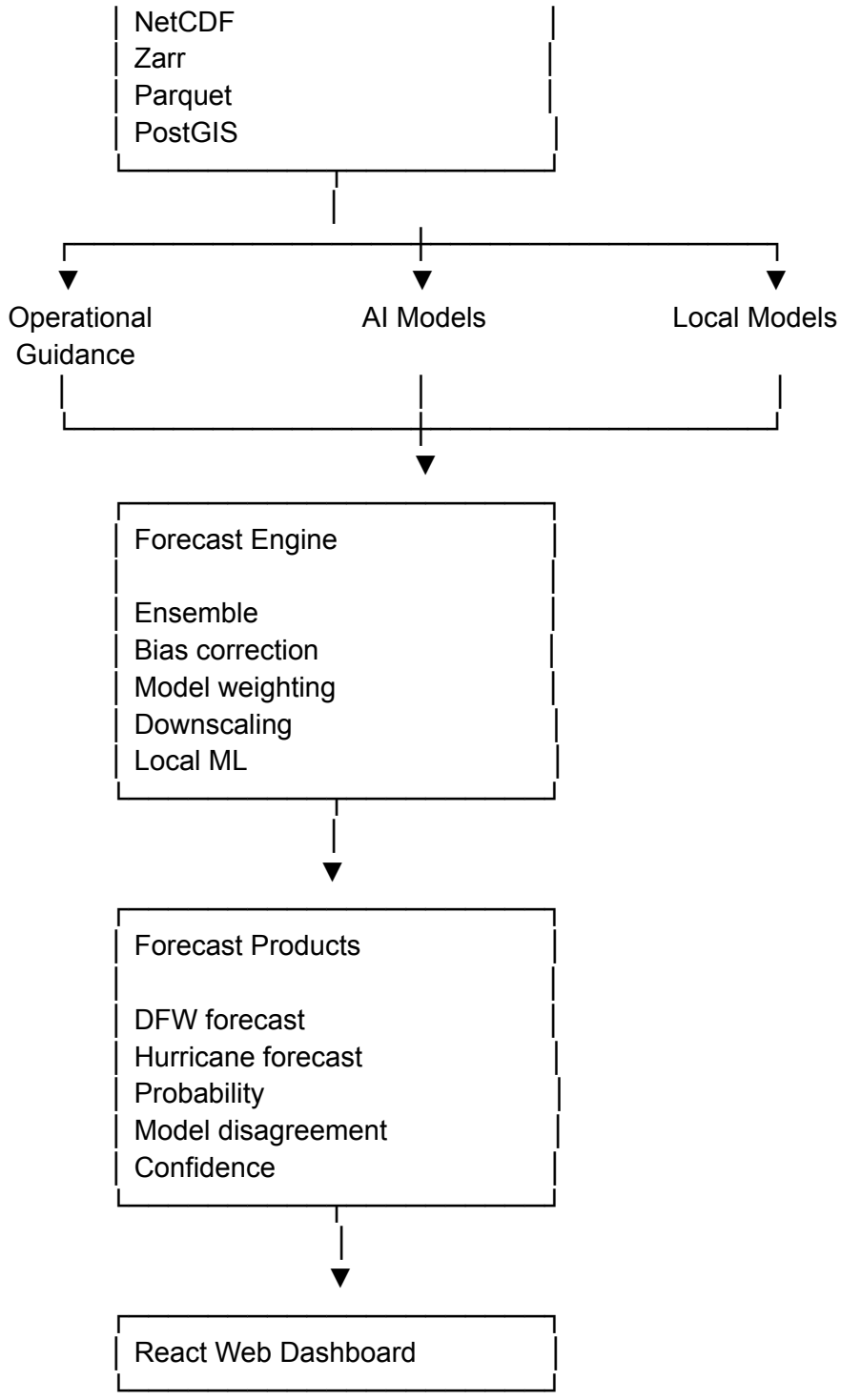
Do NOT design the system around requiring cloud GPU infrastructure.

Cloud services may be used for data acquisition, but model inference and experimentation should preferentially occur locally.

2. Core Architecture

Use a modular architecture approximately like:





3. Dashboard — DFW Weather

Create a primary dashboard centered on the Dallas–Fort Worth metropolitan area.

Default location should be the user's configured home location, initially DFW/Frisco, Texas.

The dashboard should show:

Current Conditions

- Temperature
- Feels-like temperature
- Dew point
- Relative humidity
- Wind speed/direction
- Gusts
- Pressure
- Precipitation
- Cloud cover
- Visibility
- Recent precipitation
- Radar status

Show both the latest observation and observation timestamp.

Forecast

Provide:

- Next 24 hours
- Next 48 hours
- 7-day forecast
- 10-day forecast where available

Variables:

- Temperature
- Dew point
- Relative humidity
- Precipitation probability
- Precipitation amount
- Wind
- Gusts
- Cloud cover
- CAPE
- CIN
- Lifted Index
- storm-relative parameters where relevant

Forecast Comparison

Display forecasts from multiple sources side-by-side.

At minimum:

- HRRR/RRFS
- GFS
- ECMWF IFS
- ECMWF AIFS
- available ensemble guidance
- locally run AI model(s)
- NWS/NBM guidance when available

Example:

4 PM Tomorrow

HRRR 101°F
GFS 99°F
ECMWF 100°F
AIFS 99°F
Local AI 100°F
NWS 101°F

SYSTEM FORECAST
100°F
Confidence: High

The system should explicitly expose model disagreement.

If models range from 97°F to 103°F, the user should see that uncertainty rather than a falsely precise 100°F forecast. This goes for precipitation forecasting as well.

4. Forecast Verification

This is a critical component.

The system MUST save historical forecasts and compare them against actual observations.

For every forecast, record:

- forecast initialization time

- target time
- model
- predicted value
- actual observed value
- error

Calculate:

- MAE
- RMSE
- bias
- correlation
- calibration
- forecast skill

Track this separately for:

- 1-hour
- 3-hour
- 6-hour
- 12-hour
- 24-hour
- 48-hour
- 72-hour
- 7-day

The system should continuously answer:

> Which model is currently best for DFW?

And:

> Where are the models systematically wrong?

Create a model-performance dashboard.

Example:

> DFW TEMP MAE

HRRR 1.42°F
ECMWF 1.51°F
GFS 1.87°F
AIFS 1.38°F
LOCAL MODEL 1.21°F

This verification system is more important than making the UI look pretty.

The system should never claim that the local model is better without statistically meaningful historical verification.

5. Local Forecast Model

Eventually implement a local forecasting layer whose job is NOT to replace global atmospheric models.

Instead, it should learn their errors.

Inputs may include:

- HRRR forecast
- RRFS forecast
- GFS forecast
- ECMWF forecast
- AIFS forecast
- AI model predictions
- NWS/NBM guidance
- recent observations
- historical weather
- radar-derived features
- satellite-derived features
- soil moisture
- atmospheric stability
- wind
- dew point
- pressure
- time of year
- hour of day
- geographic/location features

Output:

- corrected temperature forecast
- corrected dew point
- corrected precipitation probability
- corrected wind
- uncertainty distribution

This should initially be implemented with conventional ML models before attempting a large neural network.

Possible models:

- XGBoost
- LightGBM
- Random Forest
- neural network
- temporal transformer
- ensemble model

The system should determine empirically which performs best.

6. Real-Time Atmospheric Data

This is a major requirement.

The ingestion layer should obtain the freshest publicly available data possible.

Prioritize authoritative sources.

NOAA/NWS

Investigate and implement:

- HRRR
- RRFS
- GFS
- GDAS
- GEFS
- NBM
- NEXRAD
- METAR
- ASOS
- upper-air observations
- surface observations
- radiosondes
- GOES satellite products
- SPC products
- WPC products

NOAA's data infrastructure should be treated as a primary source.

ECMWF

Implement access to:

- IFS
- AIFS
- AIFS ensemble where available
- relevant open forecast products

ECMWF currently provides real-time open IFS/AIFS data, and AIFS data are released as forecasts are produced.

Tropical Weather

Implement:

- NHC Tropical Weather Outlook
- NHC advisories
- ATCF data
- NHC GIS feeds
- storm tracks
- forecast tracks
- cone of uncertainty
- wind probabilities
- watches/warnings
- wind fields
- storm surge products
- best track

NHC provides GIS/RSS feeds containing graphical tropical outlooks, forecast tracks, cones, watches/warnings, wind fields, and 120-hour wind probabilities.

7. Data Latency

Data freshness is a first-class feature.

Every dataset must have:

- source
- valid time
- initialization time
- download time
- processing time
- latency
- version/run identifier

The UI should show:

"HRRR initialized 12:00 UTC — downloaded 13:14 UTC — processed 13:16 UTC"

The ingestion system should automatically poll sources according to their update schedules.

Do not repeatedly download unchanged data.

Use:

- checksums
- ETags
- timestamps
- run identifiers
- file manifests

to determine whether new data are available.

The system should be event-driven/asynchronous rather than a collection of blocking scripts.

8. Local AI Weather Models

Investigate running pretrained weather models locally.

Priority candidates:

- GraphCast
- Pangu-Weather
- FourCastNet
- AIFS where technically feasible
- newer open-source AI weather models as they become available

ECMWF currently documents local execution tooling for GraphCast, Pangu-Weather and FourCastNet and explicitly recommends GPU hardware.

The system should automatically detect GPU availability and benchmark:

- VRAM usage
- inference time
- CPU utilization
- GPU utilization
- forecast generation time

Do not assume every model will fit into 16 GB VRAM.

Implement model-specific configuration and memory optimization.

Possible techniques:

- FP16

- BF16
- mixed precision
- gradient-disabled inference
- memory-efficient attention
- model quantization where scientifically appropriate

9. Hurricane Tracker

Create a dedicated Hurricane/Tropical page.

The page should have a full-screen interactive map.

Display:

- tropical disturbances
- areas of development
- tropical depressions
- tropical storms
- hurricanes
- major hurricanes

For each disturbance display:

- probability of development
- development window
- current coordinates
- movement
- pressure
- wind
- classification
- NHC discussion/advisory timestamp

For active systems display:

- observed track
- forecast track
- NHC cone
- local model track
- ensemble tracks
- wind radii
- watches/warnings
- probability of tropical-storm-force winds
- probability of hurricane-force winds

The map should support:

- animation through time
- zoom
- pan
- layer toggles
- forecast-time slider
- model selection

10. Hurricane Model Comparison

For each storm, build a model-comparison panel.

Example:

72-HOUR POSITION

NHC 28.4N / 84.2W
GFS 28.9N / 83.7W
ECMWF 28.1N / 84.5W
HAFS 28.3N / 84.0W
AIFS 28.7N / 84.1W
LOCAL ENSEMBLE 28.4N / 84.3W

Then calculate:

- consensus position
- track spread
- intensity spread
- pressure spread
- landfall probability
- landfall location distribution

Do NOT present the local model as authoritative.

Explicitly show disagreement and uncertainty.

11. Hurricane Local Forecasting

Create a future research component capable of learning hurricane forecast errors.

Potential inputs:

- NHC forecast
- HAFS
- HWRF where available

- GFS
- ECMWF
- AIFS
- ensemble guidance
- SST
- ocean heat content
- vertical wind shear
- atmospheric humidity
- upper-level divergence
- steering flow
- pressure
- satellite observations
- aircraft reconnaissance where publicly available

Potential outputs:

- track correction
- intensity correction
- uncertainty distribution

The ultimate objective is:

> Determine whether an independently trained ensemble/ML system can improve upon existing consensus guidance for selected forecast horizons.

This must be evaluated retrospectively and with strict train/test separation.

12. Local Numerical Weather Prediction

Investigate the ability to run a small regional numerical model locally.

Do NOT initially attempt to reproduce HRRR nationally.

Instead investigate:

- WRF
- MPAS
- other regional NWP frameworks

Potential architecture:

GLOBAL MODEL
↓
CONUS MODEL
↓

TEXAS MODEL
↓
DFW NEST

The local model should use larger-scale operational analysis/forecast data as boundary conditions.

Determine whether the RTX 5070 Ti can reasonably run:

- 9 km domain
- 3 km domain
- 1 km domain

over progressively smaller geographic areas.

Benchmark:

- simulation time
- real-time ratio
- GPU utilization
- memory utilization

For example:

"12-hour simulation takes 3.2 hours"

versus:

"12-hour simulation takes 22 minutes."

The objective is to determine what resolution is actually practical on the user's hardware.

13. Radar

Integrate NEXRAD data.

For DFW prioritize:

- KFWS
- surrounding radar sites
- composite reflectivity
- base reflectivity
- velocity
- correlation coefficient
- differential reflectivity
- precipitation estimates

The system should support near-real-time radar ingestion and display.

Eventually derive local features such as:

- storm motion
- precipitation intensity
- rotation
- hail signatures
- convective initiation
- storm cell tracking

A local short-term nowcasting model could eventually predict:

> Probability of measurable precipitation at the user's location in the next 15/30/60/120 minutes.

14. Data Storage

Design for large datasets from the beginning.

Use appropriate formats for scientific multidimensional data.

Consider:

- Zarr
- NetCDF
- GRIB2
- Parquet
- PostgreSQL/PostGIS
- object-style local storage

Do not put massive atmospheric grids into MongoDB simply because MongoDB is familiar.

Use the database for metadata/indexing rather than forcing it to hold every numerical array.

Example:

PostgreSQL:

- model runs
- metadata
- timestamps
- forecasts
- observations
- verification statistics

Zarr/NetCDF:

- atmospheric grids
- model fields
- historical forecast datasets

Parquet:

- ML training datasets
- point forecasts
- verification records

15. Processing Pipeline

Every data source should flow through a standardized pipeline:

INGEST
↓
VALIDATE
↓
NORMALIZE
↓
STORE
↓
DERIVE FEATURES
↓
RUN MODELS
↓
ENSEMBLE
↓
POST-PROCESS
↓
VERIFY
↓
PUBLISH

Every processing stage should be independently executable.

If one model fails, the entire forecasting system should NOT fail.

16. Dashboard Technology

Preferred architecture:

Frontend:

- React
- TypeScript
- Vite
- Tailwind
- MapLibre GL or equivalent
- charting library suitable for time-series/scientific visualization

Backend:

- Python
- FastAPI
- asyncio
- background workers

Scientific stack:

- xarray
- NumPy
- SciPy
- PyTorch
- JAX
- Dask where useful
- rasterio/geospatial tooling
- cfgrib/ecCodes
- Zarr

Database:

- PostgreSQL/PostGIS

Containerization:

- Docker
- Docker Compose initially

GPU:

- NVIDIA Container Toolkit
- CUDA

17. Forecast Confidence

Every forecast should contain uncertainty.

Never simply output:

"Tomorrow: 101°F"

Instead:

"Tomorrow's high: 100.8°F

80% interval: 98.9–102.5°F
Model spread: 2.7°F
Confidence: High"

For precipitation:

"Rain probability: 64%

Expected accumulation:
0.20–0.75 in (50% interval)

Potential:
2.1 in (95th percentile)"

For hurricanes:

"72-hour track:

Median: X
50% probability radius: X
90% probability radius: X"

The system should distinguish:

- model uncertainty
- observational uncertainty
- forecast uncertainty
- local post-processing uncertainty

18. Forecast Explainability

The dashboard should answer:

"Why does the local forecast differ from HRRR?"

Example:

LOCAL MODEL: 100°F
HRRR: 102°F

Reason:

- HRRR currently overpredicting afternoon mixing by ~1.2°F in comparable setups
- observed dew point 2°F below HRRR analysis
- soil moisture below model climatology
- ECMWF/AIFS support cooler solution

This should be data-driven rather than generated by an LLM inventing explanations.

19. Automated Forecast Cycle

The entire system should operate autonomously.

Example:

00:00 UTC
↓
Detect new model runs
↓
Download newest atmospheric data
↓
Validate
↓
Process
↓
Run AI models on GPU
↓
Generate ensemble
↓
Generate DFW forecast
↓
Generate tropical forecast
↓
Update dashboard
↓
Record forecast
↓
Wait for next data

The system should maintain a live status page showing:

- latest data received

- latest model run
- currently executing model
- GPU utilization
- queue
- processing latency
- failed downloads
- stale datasets

20. Alerts

Implement configurable alerts.

Examples:

- DFW severe weather probability exceeds threshold
- DFW temperature forecast exceeds 105°F
- freeze probability
- heavy rainfall probability
- tornado environment detected
- tropical system enters Gulf
- hurricane probability within X miles
- significant model disagreement
- forecast confidence drops sharply
- hurricane track shifts toward Texas

Alerts should be configurable and eventually support:

- browser notifications
- email
- Discord
- Telegram
- mobile/web push

21. Scientific Integrity

This is extremely important.

The system should never confuse:

"interesting forecast"

with

"accurate forecast."

Every forecasting improvement must be measurable.

Maintain a historical archive.

Use:

- rolling validation
- strict temporal train/test splits
- held-out storms
- held-out weather regimes
- baseline models

The system should continuously compare itself against:

- persistence
- climatology
- HRRR
- GFS
- ECMWF
- NWS/NBM
- consensus models
- official NHC forecast

Never tune a model using future observations.

Avoid leakage from forecast products that were generated using information unavailable at the forecast initialization time.

22. Development Roadmap

Do NOT attempt to implement the entire system simultaneously.

Phase 1:

Build the DFW dashboard.

Implement:

- observations
- HRRR
- GFS
- ECMWF
- NWS
- forecast charts
- map

- data timestamps

Phase 2:

Build the ingestion/storage architecture.

Implement:

- automatic downloads
- caching
- Zarr/GRIB handling
- metadata
- historical archive

Phase 3:

Implement forecast verification.

Store forecasts and actual observations.

Calculate model performance.

Phase 4:

Implement local GPU AI models.

Start with whichever pretrained model can reliably run within 16 GB VRAM.

Benchmark everything.

Phase 5:

Implement local DFW ML post-processing.

Train models to predict forecast error.

Phase 6:

Build the hurricane tracker.

Implement:

- NHC tropical outlook
- active storms
- tracks
- cones
- wind probabilities
- model tracks

- ensemble visualization

Phase 7:

Implement hurricane model comparison and local ML.

Phase 8:

Investigate WRF/MPAS regional modeling.

Phase 9:

Build radar nowcasting.

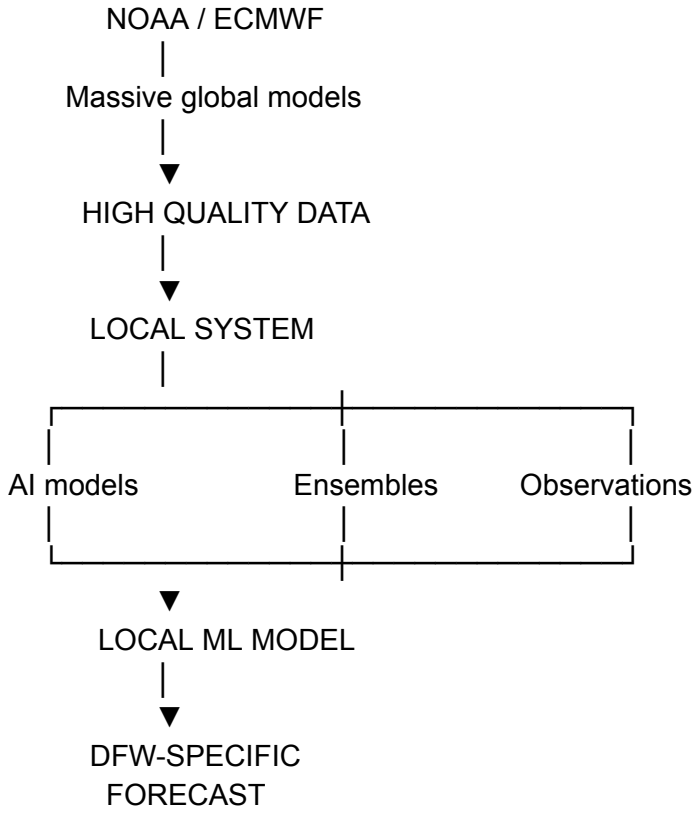
Phase 10:

Build automated alerts and continuous autonomous forecasting.

23. Important Design Principle

The system should NOT attempt to compete with NOAA by brute force.

The strategy should be:



The goal is to exploit specialization.

A national model has to forecast everywhere.

This system only needs to become extraordinarily good at forecasting:

> DFW / North Texas

while maintaining a high-quality hurricane analysis system for the Atlantic/Gulf.

24. Final Objective

The finished product should feel like a miniature personal weather operations center.

Opening the dashboard should immediately tell the user:

1. What is happening over DFW right now?
2. What will happen over DFW over the next 1–10 days?
3. What do the major forecast models disagree about?
4. What does the locally optimized forecast predict?
5. How confident is that prediction?
6. Is severe weather developing?
7. Is anything happening in the Gulf/Atlantic that could eventually affect Texas?
8. Are there active tropical systems?
9. What are the official forecasts?
10. What does the independent local forecasting system predict?
11. How well has each model been performing recently?
12. Is the local system actually outperforming the operational guidance?

The ultimate goal is not merely to display weather data.

The goal is to create a continuously running, scientifically measurable, GPU-accelerated personal weather forecasting laboratory that uses the world's best publicly available atmospheric data and models, performs additional computation locally, and learns over time where those models can be improved for the user's specific location.